

Where's Waldo and Wilma?

Object Detection with Deep Learning

Niclas Pedersen Johan Askjær

Aalborg University
Deep Learning Mini Project - Spring 2026

1 Introduction

The purpose of this mini project was to build a deep learning pipeline for detecting Waldo and Wilma in a simplified image dataset from Kaggle [1]. The task is formulated as object detection rather than image classification, since the model must both decide which character is present and localise that character with a bounding box.

The project contains two connected stages. The first stage generates YOLO-format labels for an initially unlabelled dataset. This includes estimating the target bounding box and assigning the correct class label. The second stage trains and evaluates a YOLOv8 detector using those generated labels. The label-generation stage is therefore part of the machine learning pipeline rather than only a preprocessing detail, because label errors directly affect the trained detector.

The main technical issue found during the project was the class assignment for the resized images. The original 350×500 images and resized 224×224 images contain the same scenes, so the class labels should match across resolutions. However, the first label-generation attempt produced a strongly skewed class distribution in the resized annotation file. The final workflow solves this by using the original-resolution pass as the reliable source of class identity and reusing that identity for the matching resized image, while still detecting the bounding box separately at each resolution.

The final objective was therefore to demonstrate a complete and consistent detection pipeline: automatic annotation generation, dataset conversion to YOLO format, fine-tuning of a pretrained YOLOv8 model, and evaluation using object detection metrics such as precision, recall, and mean Average Precision (mAP).

2 Data

The dataset consists of synthetic beach-scene images generated for a simplified Waldo/Wilma detection task. Positive images contain either Waldo or Wilma as the target object. Some negative/background examples are also present in the training batches and are represented as empty-label images in the YOLO dataset. This is useful because it exposes the detector to scenes where no target object should be predicted.

Two versions of the positive images are available. The original images have a resolution of 350×500 pixels, while the resized images have a resolution of 224×224 pixels. The detector is trained on the 224×224 images, since this is also the training image size used by YOLOv8. The original-resolution images are still used during annotation generation because they preserve more visual detail for the class decision.

The final positive class distribution is balanced. After the corrected label-generation pipeline, the annotation files contain 2488 Waldo labels and 2512 Wilma labels. This balance is important because a detector trained on a skewed dataset could appear to perform well while mainly learning the majority class.

The dataset was split into training, validation, and test subsets using the 70/15/15 split implemented in `train.py`. The training split shown in `labels.jpg` contains 1747 Waldo instances and 1744 Wilma instances. The training labels are therefore effectively balanced after the relabelling fix.

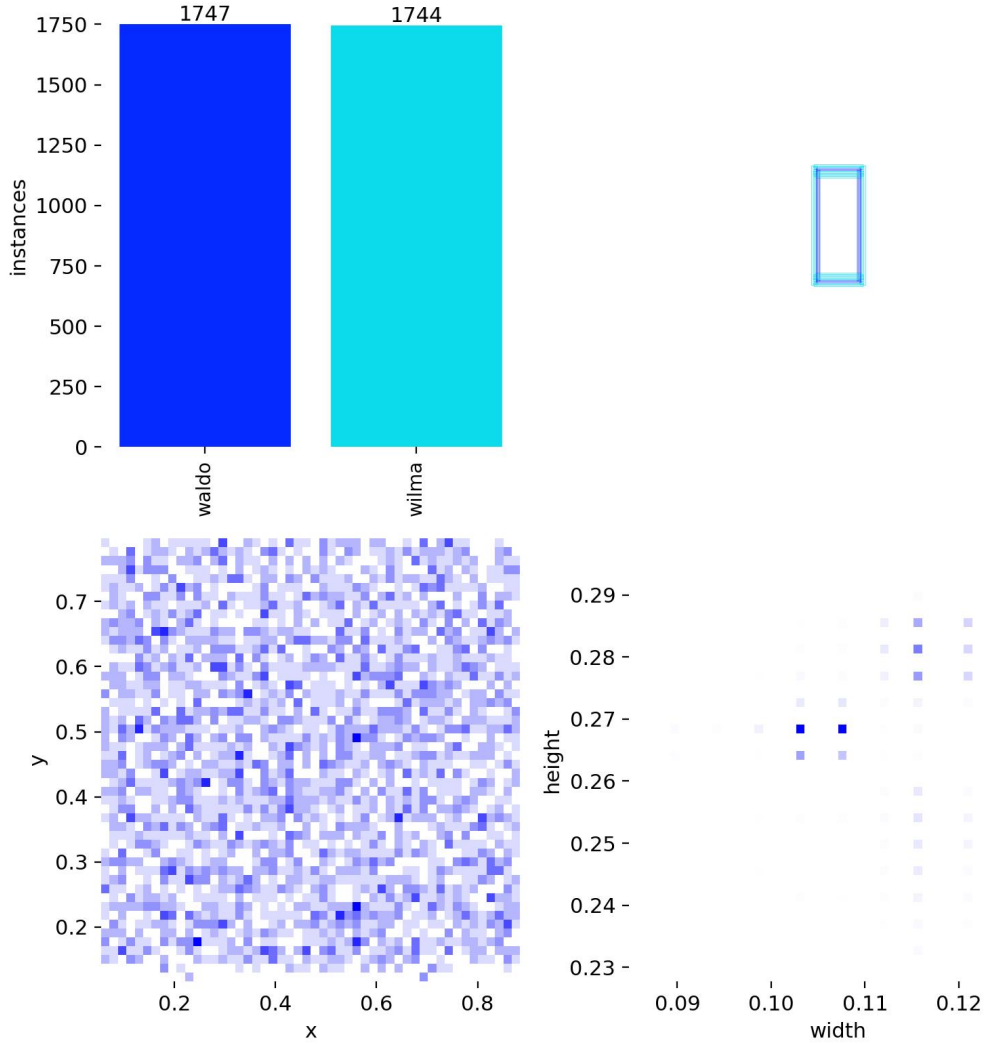


Figure 1: Training-label overview after the relabelling fix. The class counts are effectively balanced, with 1747 Waldo instances and 1744 Wilma instances. The spatial heatmap indicates that the characters are distributed across the image instead of being concentrated in one fixed location.

Figure 1 also shows the distribution of bounding-box positions and sizes. The (x, y) heatmap is spread across the image, which reduces the risk that the model can solve the task only by learning a fixed target position. The bounding-box sizes cluster around a normalised width of approximately 0.10 and a normalised height of approximately 0.27. This reflects the consistent character scale produced by the synthetic generator.

3 Method

3.1 Automatic bounding-box generation

The original dataset did not include YOLO annotations, so labels had to be generated automatically. A direct template-matching approach was first considered, but this was not reliable because the shared beach background produced strong false matches. The final localisation method therefore used image differencing against an estimated background.

A background image was computed using the per-pixel median of randomly selected images. Since the character appears at different locations across the dataset, the median operation suppresses the moving foreground character and preserves the common beach background. Each input image was then compared with the estimated background using absolute differencing. The difference image was thresholded, and morphological closing and opening were applied to remove small artefacts. The largest remaining contour

was selected as the target region, and the corresponding bounding rectangle was converted to YOLO format.

Bounding-box generation is performed independently for the original and resized images. This is necessary because the pixel coordinates change with image resolution. Class identity, however, should not change when an image is resized, which motivated the corrected class-labelling workflow described below.

3.2 Class-label generation and relabelling fix

After the target bounding box had been found, the class label had to be assigned as either Waldo or Wilma. The first implementation used the function `which_character`, which compared the detected crop against `waldo.png` and `wilma.png` using masked mean squared error (MSE). The template with the lower error was selected as the class label.

This worked reasonably well for the original-resolution images, but it failed on the resized 224×224 images. The issue is that Waldo and Wilma share a similar red-and-white shirt pattern. The most useful discriminator is the lower body: Waldo has dark blue jeans, while Wilma has a bright cyan-blue skirt. At 224×224 resolution, the character crop is small, and the cyan skirt information is partly averaged away by resizing. As a result, the classifier tended to assign too many resized crops to Waldo.

Table 1: Class-count checks during label generation.

Annotation pass	Waldo	Wilma	Interpretation
Original 350×500 labels	2488	2512	Balanced and reliable
Resized 224×224 labels before fix	3160	1840	Incorrect skew toward Waldo
Resized labels with bottom-half MSE	3058	1942	Improved, but still skewed
Final resized labels with class lookup	2488	2512	Consistent with original labels

The first fix was to compare only the bottom half of each detected crop with the bottom half of the templates. This removed much of the shirt region from the MSE calculation and made the comparison focus more on the trousers/skirt region. This improved the original-resolution classification but did not fully solve the resized-image problem.

The final fix was to reuse class identities across resolutions. The original and resized folders contain matching images, for example `image_N.png` and `resized_image_N.png`. The original-resolution pass is run first and produces a mapping from image stem to class name. The resized pass then uses this mapping through the `class_lookup` argument in `process_folder`. The bounding box is still detected directly on each resized image, but the class identity is copied from the reliable original-resolution result.

Algorithm 1 Corrected label-generation workflow

- 1: Estimate shared background using a median image
 - 2: Process original 350×500 images
 - 3: **for** each original image **do**
 - 4: Detect bounding box using background differencing
 - 5: Classify crop using bottom-half template MSE
 - 6: Store mapping from original image stem to class name
 - 7: **end for**
 - 8: Convert original stems to matching resized stems
 - 9: Process resized 224×224 images
 - 10: **for** each resized image **do**
 - 11: Detect bounding box using background differencing
 - 12: Reuse class identity from the original-resolution lookup
 - 13: Write YOLO label file
 - 14: **end for**
-

This correction is important because the detector is trained on the resized images. If the old direct low-resolution template classifier is used, many Wilma examples are incorrectly labelled as Waldo. The detector would then be trained on label noise, and poor class predictions could be caused by the annotation pipeline rather than by the model architecture.

3.3 Generation of negative samples

To expose the detector to scenes that contain neither Waldo nor Wilma, 500 synthetic negatives were generated by `generateNegativeSamples.py`. The shared background was first estimated as the per-pixel median of 50 randomly selected resized images, the same construction used during bounding-box generation. Each negative was then produced by perturbing this background with three independent augmentations: additive Gaussian noise, a global brightness offset, and a contrast factor in applied around the image mean. The resulting images were clipped to $[0, 255]$ and saved with empty YOLO label files so that they act as background examples during training. The 500 negatives were combined with the positive resized images and split jointly under the 70/15/15 partition, which spreads negatives across all three splits.

3.4 YOLOv8 detector

The detector was trained using YOLOv8s, starting from the pretrained `yolov8s.pt` checkpoint. YOLOv8 was chosen because it is a one-stage detector that directly predicts both bounding boxes and class probabilities, which matches the project task. According to the Ultralytics model description, YOLOv8s is the small variant with approximately 11 million parameters and 28.6 billion flops [2]. The present project uses a smaller input resolution than the standard 640×640 benchmark setting, so the computation per image is lower while the parameter count remains the same.

Table 2: Training configuration used for the final YOLOv8s run.

Setting	Value
Base model	<code>yolov8s.pt</code>
Epochs	30
Image size	224
Batch size	32
Optimizer	<code>auto</code> (resolved to SGD)
Initial learning rate	0.01
Final learning-rate factor	0.01
Momentum	0.937
Weight decay	0.0005
Warmup	3 epochs
Mixed precision	Enabled
Seed	0, deterministic
Split	70/15/15
Approximate training time	490 s

The final training run used the standard YOLO augmentation pipeline, including HSV jitter, scale augmentation, horizontal flipping, mosaic augmentation, and random erasing. Mosaic augmentation was disabled for the final 10 epochs using `close_mosaic=10`. This lets the model benefit from strong augmentation early in training while ending training on images that are closer to the actual validation distribution.

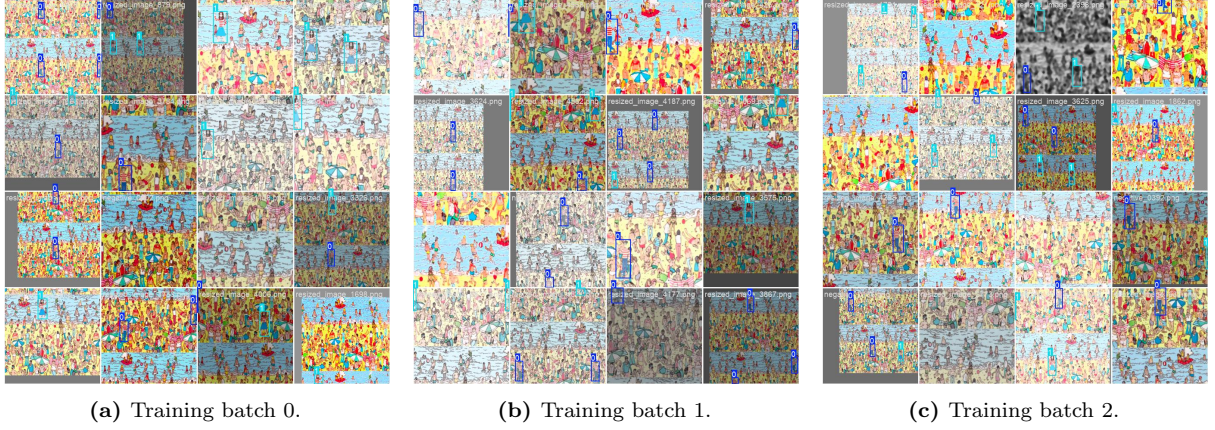


Figure 2: Example training batches generated by the YOLO training pipeline. The examples show positive Waldo/Wilma annotations as well as empty-label negative images.

4 Experiments and Results

4.1 Training behaviour

Figure 3 shows the training curves from the final run. The three training losses decrease consistently over the full 30 epochs. The validation losses follow the same general trend, which indicates stable optimisation and no clear divergence between training and validation performance.

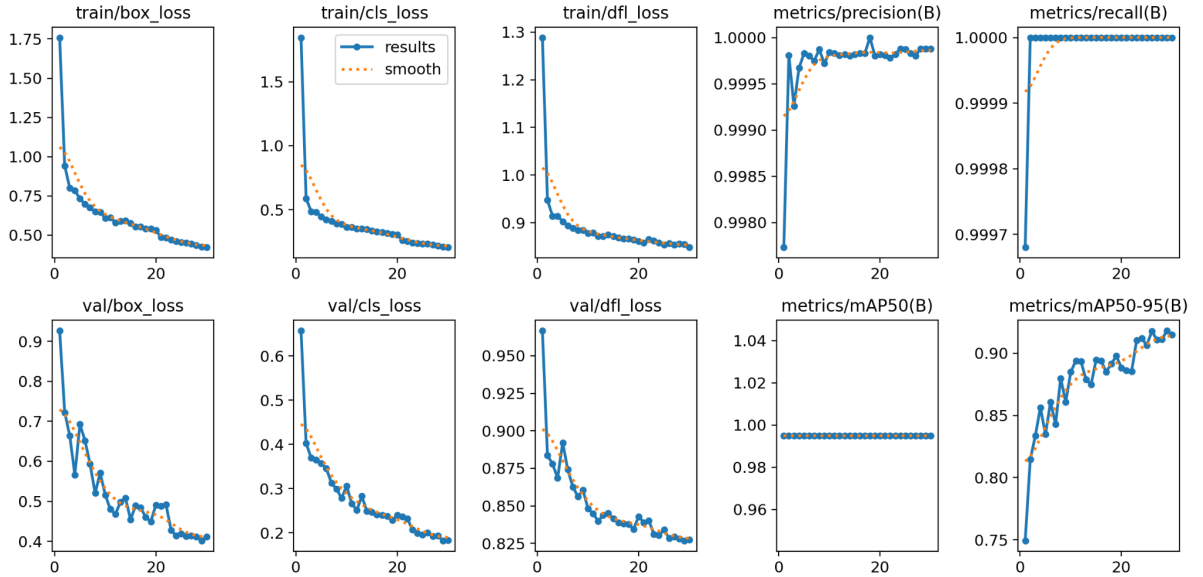


Figure 3: YOLOv8s training results over 30 epochs. The loss curves decrease for both training and validation data, while precision, recall, and mAP remain high after the relabelling fix.

The final validation metrics are shown in Table 3. The model reaches nearly perfect precision and recall on the validation split, with $\text{mAP}@0.5$ saturating at 0.995. The stricter $\text{mAP}@0.5:0.95$ value is 0.91505, which shows that the predicted boxes are not only detected but also reasonably well localised under stricter IoU thresholds.

Table 3: Headline validation metrics at epoch 30.

Metric	Value
Precision(B)	0.99988
Recall(B)	1.00000
mAP@0.5	0.99500
mAP@0.5:0.95	0.91505
Validation box loss	0.41198
Validation classification loss	0.18354
Validation DFL loss	0.82741

The loss reduction from epoch 1 to epoch 30 is summarised in Table 4. The classification loss falls strongly for both the training and validation splits. This is consistent with the corrected class labels, since the model is no longer trained on a resized dataset where many Wilma examples are incorrectly labelled as Waldo.

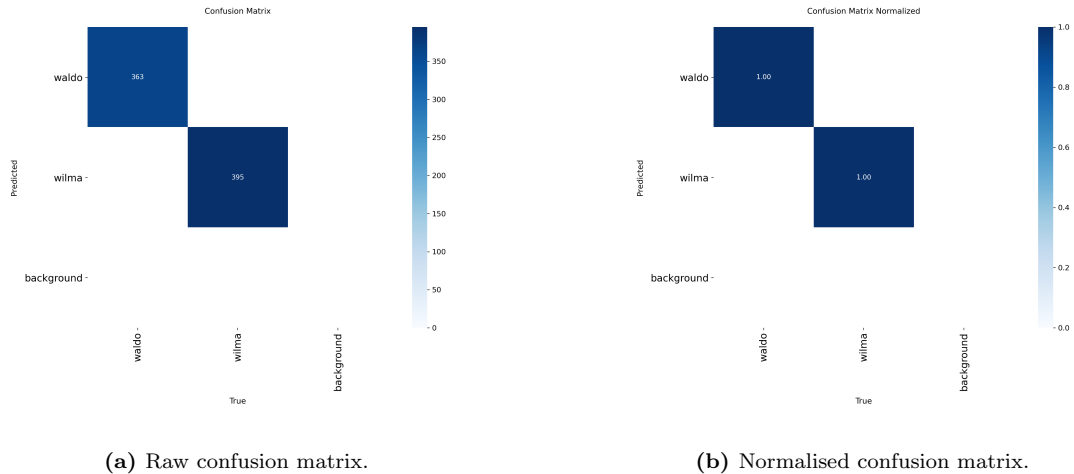
Table 4: Loss reduction from epoch 1 to epoch 30.

Loss	Training	Validation
Box loss	1.759 $\rightarrow$ 0.422	0.926 $\rightarrow$ 0.412
Classification loss	1.847 $\rightarrow$ 0.205	0.657 $\rightarrow$ 0.184
DFL loss	1.289 $\rightarrow$ 0.848	0.967 $\rightarrow$ 0.827

A useful detail is that mAP@0.5 is already saturated early in training, while mAP@0.5:0.95 continues improving from 0.749 at epoch 1 to 0.915 at epoch 30. This means that the easy version of the detection task is solved quickly, but the stricter localisation quality still improves throughout training.

4.2 Confusion matrix

The confusion matrix is the clearest sanity check that the relabelling fix worked. The earlier label-generation pipeline produced a strong Waldo/Wilma imbalance in the resized annotations. After the fix, the validation confusion matrix is perfectly diagonal for the positive classes.

**Figure 4:** Validation confusion matrices for the final YOLOv8s run. The validation split contains 363 true Waldo instances and 395 true Wilma instances, all classified correctly.

The raw matrix contains 363 correctly predicted Waldo instances and 395 correctly predicted Wilma instances, with zero off-diagonal entries. The normalised matrix therefore reads 1.00 on the diagonal for both classes. This result indicates that the final training labels are consistent enough for the model to learn both classes cleanly.

4.3 Qualitative validation examples

Figures 5 and 6 compare validation labels with the corresponding predictions. The predicted boxes generally match the labelled target locations, and the class labels agree visually with the target character. The high confidence values in the prediction figures are consistent with the near-perfect precision and recall reported in Table 3.

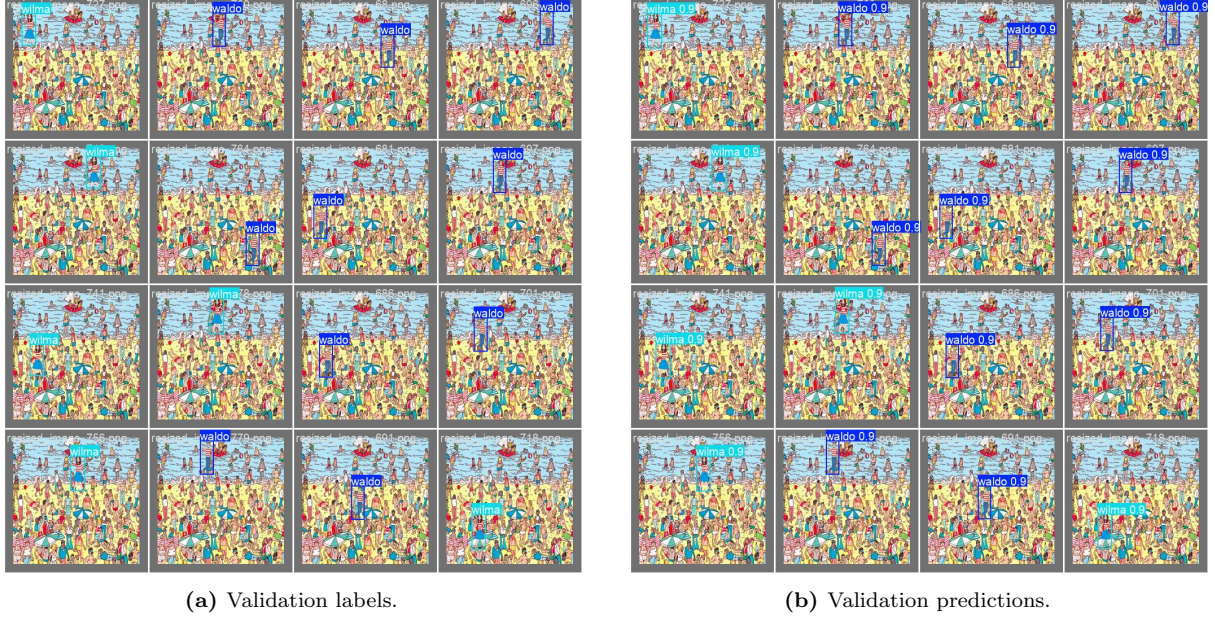


Figure 5: Validation batch 0: ground-truth labels compared with model predictions.

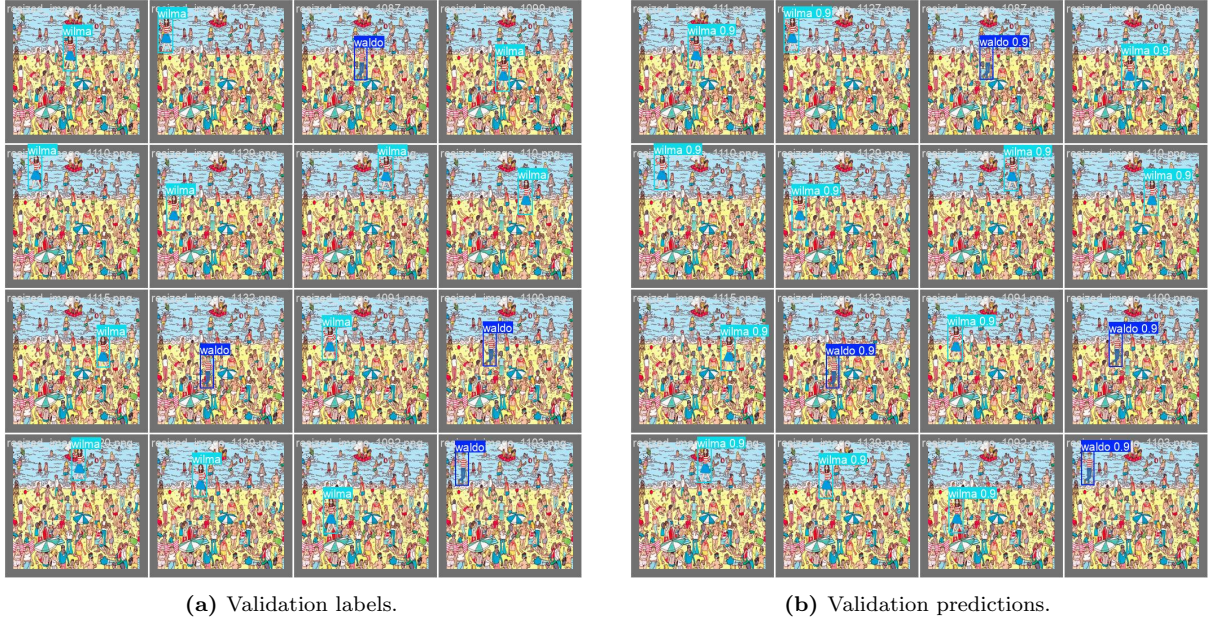


Figure 6: Validation batch 1: ground-truth labels compared with model predictions.

The qualitative examples also show why the task is still meaningful despite the high numerical results. The target characters are small relative to the image and appear in cluttered scenes with many visually similar distractors. At the same time, the scenes are still generated from the same synthetic distribution, so the results should be interpreted as strong performance on the controlled benchmark rather than proof of general performance on arbitrary Where’s Waldo book scans.

5 Discussion

The final model results are much stronger than the earlier run because the main source of error was corrected before training. The first resized annotation file contained 3160 Waldo labels and 1840 Wilma labels, even though the original-resolution labels were balanced. This would bias the detector toward Waldo and make the validation predictions difficult to interpret. The corrected pipeline gives a balanced training distribution of 1747 Waldo and 1744 Wilma instances, which is a better foundation for training.

The loss curves do not show the typical pattern of overfitting. Training and validation losses decrease together, and the final validation classification loss is slightly lower than the final training classification loss. The stricter $\text{mAP}@0.5:0.95$ also continues improving across the run, which suggests that the detector is improving localisation quality rather than only memorising labels.

However, the results should still be interpreted with the dataset limitations in mind. The validation and test images come from the same generator as the training images. Therefore, the validation split tests generalisation to unseen samples from the same synthetic distribution, not generalisation to real Where's Waldo images or book scans. A stronger out-of-distribution test would be to evaluate the trained model on a real or manually prepared Waldo scene that was not generated by the same image pipeline.

Another important point is that $\text{mAP}@0.5$ saturates at 0.995 very early. This is not necessarily a problem. It mainly shows that the coarse localisation task is easy for this dataset. The more informative metric is $\text{mAP}@0.5:0.95$, because it evaluates the box quality across stricter IoU thresholds. This metric continues to improve and reaches 0.915 at the end of training.

6 Conclusion

This mini project presented an end-to-end YOLOv8 pipeline for detecting Waldo and Wilma in synthetic beach-scene images. The project first generated YOLO-format annotations automatically. Bounding boxes were produced using background differencing, while class labels were corrected by transferring class identities from the original-resolution images to the corresponding resized images.

The corrected label-generation pipeline fixed the main problem in the earlier workflow. Direct class assignment on the 224×224 images produced a skewed resized annotation file with 3160 Waldo labels and 1840 Wilma labels. Bottom-half MSE improved the result but did not fully solve the low-resolution issue. The final cross-resolution class lookup made the resized annotation file match the original-resolution labels, giving 2488 Waldo labels and 2512 Wilma labels.

After training YOLOv8s for 30 epochs, the final model reached a validation precision of 0.99988, recall of 1.00000, $\text{mAP}@0.5$ of 0.99500, and $\text{mAP}@0.5:0.95$ of 0.91505. The validation confusion matrix contains 363 correctly predicted Waldo instances and 395 correctly predicted Wilma instances, with zero off-diagonal entries. This shows that the corrected labels allow the detector to learn both classes cleanly.

The project can therefore be considered successful on the controlled synthetic benchmark. The most important future improvement is to test the model on images outside the generator distribution, for example a real Where's Waldo image or a manually annotated scene with different background structure, scale, and distractors.

References

- [1] Shesh Gupta. Waldo and wilma dataset, 2026. URL <https://www.kaggle.com/datasets/sheshngupta/waldowilma>. Accessed: 2026-04-13.
- [2] Ultralytics. YOLOv8 documentation, 2026. URL <https://docs.ultralytics.com/models/yolov8/>. Accessed: 2026-04-13.